

Sals3 Marketplace

UI/UX and Build Specification

Final — for the new build

Document type	Design and build specification
Project	Sals3 marketplace, new build
Date	1 August 2026
Language standard	ASD-STE100 Simplified Technical English
Platform decision	New system. The team does not use WooCommerce.
Status	Final

READ THIS FIRST

This document uses Simplified Technical English. Each sentence gives one instruction or one fact. Sentences are short. The document uses the active voice.

Section 2 defines every technical word. Read section 2 before you read the other sections.

Section 20 gives the build order. Section 21 gives the realistic expectations for time, effort, and results.

Contents

1	About this document
2	Glossary
3	Decisions
4	The main design rule
5	Design rules for price
6	Design rules for search and filters
7	Design rules for the product page
8	Design rules for the cart and the checkout
9	Design rules for trust
10	Design rules after the purchase
11	The visual system
12	Performance rules

13	Accessibility rules
14	Forbidden patterns
15	Screen layouts
16	System architecture
17	The price and promotion engine
18	Data migration from the old system
19	Tests and launch gates
20	How to build the system from the start
21	What to expect
22	References

1. About this document

1.1 Purpose

This document tells the team how to design and how to build the new Sals3 marketplace. It gives design rules, screen layouts, code contracts, a build order, and test gates.

1.2 Who reads this document

Reader	Sections to read
Designer	Sections 3 to 15
Frontend developer	Sections 4 to 16, and sections 19 to 21
Backend developer	Sections 16 to 21
Project owner	Sections 1, 3, 20, and 21
Test engineer	Sections 12, 13, 14, and 19

1.3 How to read the rules

The document uses three words for each rule. Each word has one meaning only.

- **Must** — the team builds this. The team cannot remove it.
- **Must not** — the team does not build this. There is no exception.
- **Can** — the team makes a choice. Both choices are correct.

1.4 Language standard

This document follows ASD-STE100 Simplified Technical English. The rules below apply to all project documents.

- One sentence gives one instruction.
- A procedure sentence has 20 words or less. A description sentence has 25 words or less.
- The document uses the active voice.
- A paragraph has 6 sentences or less.
- Each technical word has one meaning. Section 2 gives the meaning.

SCOPE OF THIS DOCUMENT

This document covers the design of the screens, the contracts for the code, and the build order. It does not cover the business plan. It does not cover the marketing plan. The owner approves those plans.

2. Glossary

These are the technical words in this document. Each word has one meaning.

Word	Meaning
Accessibility	The design lets a person with a disability use the product.
Attribute	A property of a product. Examples: colour, size, material.
BFF (Backend For Frontend)	A server that collects data from many services. It gives one answer to the app.
Cart	The list of products that the buyer selects before payment.
Checkout	The screens where the buyer gives an address and pays.
COD (Cash on Delivery)	The buyer pays cash to the courier at the door.
Contrast ratio	A number that compares text brightness to background brightness. A high number is easier to read.
Conversion rate	The percentage of visitors who buy.
Cursor	A code that tells the server where the previous list of products stopped.
Dark pattern	A design that makes the buyer do an action that the buyer does not want.
Design token	A named value for a colour, a space, or a text size. The design files and the code use the same name.
Facet	One filter group on a list page. Example: Brand, Price, Size.
Fulfillment leg	One shipment from one warehouse to the buyer. One order can have many legs.
Idempotency key	A unique code on a request. The server runs the request one time only, even after a retry.
Landed price	The full amount that the buyer pays. It includes the item, the tax, and the shipping.
LCP (Largest Contentful Paint)	The time until the largest item on the screen appears.
INP (Interaction to Next Paint)	The time between a tap and the screen response.
Minor unit	The smallest unit of money. One peso has 100 minor units.
PDP (Product Detail Page)	The page for one product.
Quote	A price answer from the server. It has a version number and an expiry time.
RTS (Return to Sender)	The courier returns the parcel because the delivery failed.
SKU (Stock Keeping Unit)	A code for one exact product variant. Example: red shirt, size L.
Skeleton screen	A grey shape that shows the position of content before the content loads.
Slug	The text part of a web address. Example: <code>solar-wall-lamp</code> .
State preservation	The app remembers the scroll position and the filters when the buyer goes back.
Variant	One version of a product. Example: the blue version, size M.

Vertical slice	One complete path through all layers. Example: a category page to a paid order.
Voucher	A discount code or a discount rule.
WCAG	The international accessibility standard for the web.

3. Decisions

This table is the decision record. The team builds the ADOPT items. The team does not build the REJECT items. The team builds a DEFER item only after a new approval.

Item	Decision	What the team does
New system, not WooCommerce	ADOPT	The team builds new services. WooCommerce is the source of the old data only. See section 18.
One final price on every screen	ADOPT	One pricing service calculates the item price, the discount, the tax, and the shipping. All screens show the same total.
Group the cart by fulfillment leg	ADOPT	Group by the shipment, not by the seller name. One seller can send two shipments. Each group shows its own shipping fee and its own delivery date.
Semantic design tokens	ADOPT	Build the token layer in week one. See section 11.
State preservation on lists	ADOPT	The buyer returns from a product page to the same position with the same filters. See section 6.4.
Guest checkout	ADOPT	The buyer completes a purchase without an account. The system offers an account after the order.
Automatic voucher selection	TEST	Build a deterministic rules engine. Show the buyer which vouchers apply. Never write the word "maximum" unless the engine tests every allowed combination. See section 17.
Dark mode	TEST	Prepare the tokens. Do not make dark mode a launch requirement.
Semantic search with vectors	DEFER	First clean the categories, the attributes, and the synonyms. First measure the zero-result rate. Add vector search later.
3D and AR product media	DEFER	The sellers supply photographs only. Nobody makes 3D files for the catalogue. Images stay the default.
Feed that changes with the time of day	DEFER	This needs consent, measurement, and traffic volume. Wait.
Swipe-to-pay as the primary control	REJECT	A drag gesture lowers the completion rate. It also fails WCAG 2.5.7. Use a tap control. A drag can be an extra option only.
Glass effect on many surfaces	REJECT	Use the glass effect on two surfaces only: the top header and the bottom bar. See section 11.6.
False urgency and false scarcity	REJECT	See section 14.

4. The main design rule

THE RULE

Show many products. Show one clear priority on each screen.

4.1 Why this rule exists

Some designers remove content to make a screen clean. This is a mistake for Sals3. Sals3 buyers look for low prices. A screen with few products looks empty. An empty screen makes the buyer think that the shop has no stock. An empty screen also makes the buyer think that the prices are high.

The problem on large marketplaces is not the quantity of content. The problem is that 6 elements have the same visual weight. The buyer cannot see which element is important.

4.2 How to apply the rule

- Keep a high product density. Show 2 product columns on a telephone.
- Give one element the strongest visual weight on each screen. Example: the price on a product card.
- Make every other element quieter. Use a smaller size, a lighter weight, or a grey colour.
- Do not put two strong colours on one screen.
- The product photograph is the most colourful item on the screen. No interface element competes with it.

5. Design rules for price

Price transparency is the most important advantage of Sals3. Large platforms make the buyer calculate the price. Sals3 shows the price.

5.1 The one-number rule

- The system **must** show one final price. This is the amount that the buyer pays.
- The product card, the product page, and the cart **must** show the same number.
- The price **must not** increase at the checkout. A price increase at the end destroys trust.
- The server calculates the price. The app **must not** calculate the price.

5.2 The price block on the product page

P1,240	<- final price, largest text on the page	
P1,890	-34%	<- old price with a line through it, grey, smaller
[v] All discounts are applied <- the buyer taps to open the list		
Item price	P1,890	
Seller discount	-P450	
Platform voucher	-P200	
Shipping	P0	(you save P79)

You pay	P1,240	

- The list is closed at the start. The buyer taps to open it.
- The list shows every line. The buyer sees the arithmetic.
- Each line names the source of the discount.

5.3 Shipping cost

CRITICAL RULE

The system must show the shipping cost early. Buyers stop the purchase when the shipping cost appears at the last step.

- The system **must** ask for the delivery region one time. Put the control in the header.
- The system **must** remember the region for the next visit.
- The product card **must** show the shipping cost when the system knows the region.
- The product page **must** show a delivery date. Write a real date. Example: "Arrives 4 to 6 September".
- Do **not** write "3 to 5 business days". A date is easier to understand than a duration.

6. Design rules for search and filters

Search and filters move more revenue than any other part of the interface. Build this part early.

6.1 The search box

- Put the search box in the header. The header stays on the screen.
- Show the recent searches when the buyer taps the box.
- Show the products that the buyer looked at before.

6.2 Zero results

RULE

An empty result page is a failure. The system must always show products.

- Remove one word from the query. Then search again.
- Show this message: "No exact match for *waterproof hiking boots size 45*. These products are similar."
- Record every zero-result query. The team uses this list to correct the synonyms.

6.3 Filters

- The filter groups **must** come from the category. Do not use the same filters for all products.
- Do **not** show a "screen size" filter on a clothing page.
- Each filter value **must** show a count. Example: Nike (243).
- A filter value with 0 products **must** stay visible. Make it inactive. Do not remove it. A list that changes its length confuses the buyer.
- The selected filters **must** stay visible at the top of the list. Use small chips.
- Each chip has its own remove control. One more control clears all chips.

- The sort control is separate from the filter control. The buyer changes the sort order with one tap.
- A price filter **must** show a bar chart of the price distribution. Then the buyer sees the range before the buyer moves the control.

6.4 State preservation

THE RULE THAT MATTERS

The buyer opens a product. The buyer goes back. The list shows the same position, the same filters, and the same sort order.

The team can use pagination, a Load More control, or an infinite scroll. All three can pass. The conditions below decide the result, not the mechanism.

The design passes when all 6 conditions are true:

1. The web address holds the filters, the sort order, and the position.
2. The buyer can copy the web address. Another person opens the same list.
3. The back control returns the buyer to the same scroll position.
4. The sort order is deterministic. The same query gives the same order every time.
5. A search engine can read every page of the list.
6. The buyer can reach the page bottom. An infinite scroll that hides the bottom fails this condition.

7. Design rules for the product page

7.1 The first screen

The first screen answers 4 questions. It answers no other question.

1. What is this product?
2. What is the true price?
3. When does it arrive?
4. Can I trust the seller?

7.2 The order of the sections

No.	Section	Rules
1	Photographs	5 to 7 images. The buyer swipes. The buyer can zoom. No 3D model.
2	Title and rating	Show the rating, the quantity of reviews, and the quantity sold.
3	Price block	See section 5.2.
4	Variant selector	Use image swatches or colour swatches. Do not use a dropdown list. An unavailable variant stays visible and inactive.
5	Delivery	Show a real arrival date and the shipping fee.
6	Seller card	Show the name, the rating, the reply rate, and the verification mark.

7	Return policy	One sentence. Do not use a legal paragraph.
8	Reviews	See section 7.3.
9	Related products	This is the last section. It is never between sections 1 and 8.

COMMON ERROR

Large platforms put a related-products carousel in the middle of the product information. This interrupts the buyer. Sals3 completes the product information first.

7.3 Reviews

Most marketplaces do this part badly. A good review section is a large advantage.

- The buyer can filter the reviews by star count.
- The buyer can show only the reviews with a photograph.
- The buyer can show only the reviews for one variant. Example: "Show reviews for Size L, Black".
- The system shows summary chips above the reviews. Examples: True to size (48) , Fast delivery (31) , Thin fabric (12) .

The summary chips help the buyer to select the correct variant. This lowers the return rate.

7.4 The action bar

- The bar stays at the bottom of the screen.
- It has two controls: "Add to Cart" is secondary. "Buy Now" is primary.
- Both controls are tap controls. Do **not** use a drag control.
- After a tap, a small message appears. The buyer stays on the same page.

8. Design rules for the cart and the checkout

8.1 The cart

- Group the items by fulfillment leg. One group is one shipment.
- Each group shows its own shipping fee and its own arrival date.
- Each group shows a progress line for free shipping. Example: "Add P120 more for free shipping". This message is true. It is not a false urgency message.
- The order total stays visible when the buyer edits the cart.

8.2 The checkout

- Use one page. Show the steps as closed summary rows.
- Open one step at a time.
- The buyer **must** be able to buy without an account.
- Cash on Delivery is a main payment option. Do not put it at the bottom of a long list.
- Keep the quantity of visible form fields low. Ask only for the data that the delivery needs.

- Check each field after the buyer leaves the field. Do not show an error during the typing.
- An error message appears below its field. Use a colour and an icon. Colour alone is not sufficient.
- The last control shows the exact amount. Example: `Place Order` • `P1,319` .

8.3 Price changes during the checkout

A price can change while the buyer completes the form. The system must tell the buyer.

- The server sends a quote with a version number.
- The system compares the version at the payment step.
- If the price changed, the system shows the changed lines. The buyer approves the new total.
- The system **must not** charge a different amount without an approval.

9. Design rules for trust

Nobody knows the new Sals3. Every page must answer this question: "Is this shop safe?"

- Show the seller verification mark. The buyer taps it. It opens the join date, the order count, and the verification source.
- Show the return policy on the product page. Do not hide it in a footer link.
- Show buyer photographs in the reviews. Give them a high position.
- Show a clear support control on every order screen.
- Show the merchant identity and the price conditions. Philippine law requires this. See section 22.

10. Design rules after the purchase

This part builds buyer loyalty. Most marketplaces ignore it.

- Show a delivery timeline with real dates. Do not show a status code.
- Send a message at each status change.
- Let the buyer start a return without a telephone call. The buyer selects the item, selects the reason, and gets the label.
- Let the buyer repeat an old order with one tap.

WHY THIS MATTERS

A difficult return process is the main reason that a buyer does not come back. A simple return process costs less than a new advertisement.

11. The visual system

11.1 Design tokens

A design token is a named value. The design files and the code use the same name. Build 3 levels.

Level	Example name	Rule
1. Primitive	<code>color.teal.500</code>	A raw value. It has no meaning. Nobody uses it directly in a component.
2. Semantic	<code>color.action.primary</code>	A purpose. It points to a primitive. Components use this level.
3. Component	<code>button.primary.background</code>	One component only. Use this level only when level 2 is not sufficient.

This structure lets the team change a colour one time. All components change together.

11.2 Text

- Use one sans-serif typeface. Load 2 weights only.
- Use 5 text sizes or less.
- The price is the heaviest text on a product card.

11.3 Layout grid

- Use an 8-point spacing scale. Use 4 points for small gaps.
- Show 2 product columns on a telephone. Three columns make the photographs too small.

11.4 Colour

- Use one brand colour. Use it for actions only.
- Use one deal colour, for example red. Use it for discounts and for sale marks only.
- Do **not** use the brand colour on a promotion banner. Then the buyer cannot find the buy control.
- Make all other elements neutral grey.
- Text must have a contrast ratio of 4.5 to 1 or more.
- Colour alone must not carry information. Add an icon or text.

11.5 Dark mode

Dark mode is optional. Prepare the tokens. Build it later.

- Do not use pure black. Start the background at `#121212`.
- Show depth with lighter surface colours. Shadows are not visible on a dark background.
- Lower the saturation of the brand colour by 10 to 20 percent.

11.6 Glass effect

- Use the glass effect on 2 surfaces only: the top header and the bottom bar.
- Do **not** use it on cards, sheets, or dialogs. The effect is expensive for the processor.
- Add a solid tint behind the blur. Then the text keeps its contrast ratio.
- Add a 1-pixel border. Then the buyer sees the edge of the surface.
- Give a solid colour alternative for a device that cannot do the blur.

11.7 Motion

- Most animations last 150 to 250 milliseconds.

- Use spring motion only for a drag interaction, for example a sheet that the buyer pulls.
- Use a simple ease-out curve for all other animations.
- Obey the system setting for reduced motion.

11.8 Loading

- Use skeleton screens. Do not use a rotating circle.
- Give every image container a fixed aspect ratio. Then the layout does not move after the load.

NOTE

A layout that moves during the load makes a new site look unprofessional. Buyers do not trust it.

12. Performance rules

The typical Sals3 buyer has a low-cost Android telephone and a slow mobile connection. Every design decision costs processor time and battery.

- The team **must** test on one low-cost Android telephone. Use a real device, not a simulator.
- The team **must** test on a slow network profile.
- Set a JavaScript budget for each route. Set an image budget for each route.
- Measure LCP and INP from real buyers. Use the 75th percentile.
- Load images first. Load video later.
- Do **not** load a 3D library on a route that does not use it.
- Send images in a modern format. Send a size that matches the screen.

13. Accessibility rules

- Use correct HTML elements. A button is a `<button>` element.
- Every control must show a visible focus mark.
- A buyer must be able to complete a purchase with a keyboard.
- A touch target must be 44 by 44 pixels or more.
- Every financial action and every destructive action needs a control that is not a drag gesture.
- Announce a cart change to a screen reader.
- Give every product image a text description.

14. Forbidden patterns

DO NOT BUILD THESE

Pattern	Reason
---------	--------

A countdown timer that restarts	The message is false. It is a legal risk.
"Only 1 left" when the stock is higher	The message is false.
A dialog that the buyer cannot close	It removes buyer control.
A refusal control with shaming text	Example: "No thanks, I like to pay more." This damages the brand.
A pre-selected extra product	The buyer pays for an item that the buyer did not select.
A price that increases at the last step	This is the largest cause of an abandoned cart.
A spin-to-win game at the application start	It blocks the buyer goal.
A forced account before the purchase	It stops many buyers.

These rules also lower the legal risk under Philippine Republic Act 11967.

15. Screen layouts

15.1 Home screen

Element	Content	Behaviour
Header	Search box, delivery region, cart count	It stays at the top. It becomes smaller when the buyer scrolls down.
Category row	8 to 10 category icons	Horizontal scroll. It shows the true top categories, not a fixed list.
Deal section	Products with a real discount	Show the discount percentage. Show a real end time or no time.
Product grid	2 columns	Each card shows the photograph, the title, the final price, the old price, the rating, and the shipping cost.
List control	Load More control	See section 6.4 for the 6 conditions.
Bottom bar	Home, Categories, Cart, Orders, Account	It stays at the bottom. It uses the glass effect.

15.2 List screen

Element	Content	Behaviour
Filter chips	The selected filters	They stay below the header during the scroll. Each chip has a remove control.

Filter sheet	Facets from the category	Each value shows a count. A 0-count value stays visible and inactive.
Sort control	Relevance, price, rating, new	One tap. It is separate from the filter sheet.
Result count	"1,240 products"	It updates with the filters.
Zero-result panel	A message and similar products	It never shows an empty page.

15.3 Product page

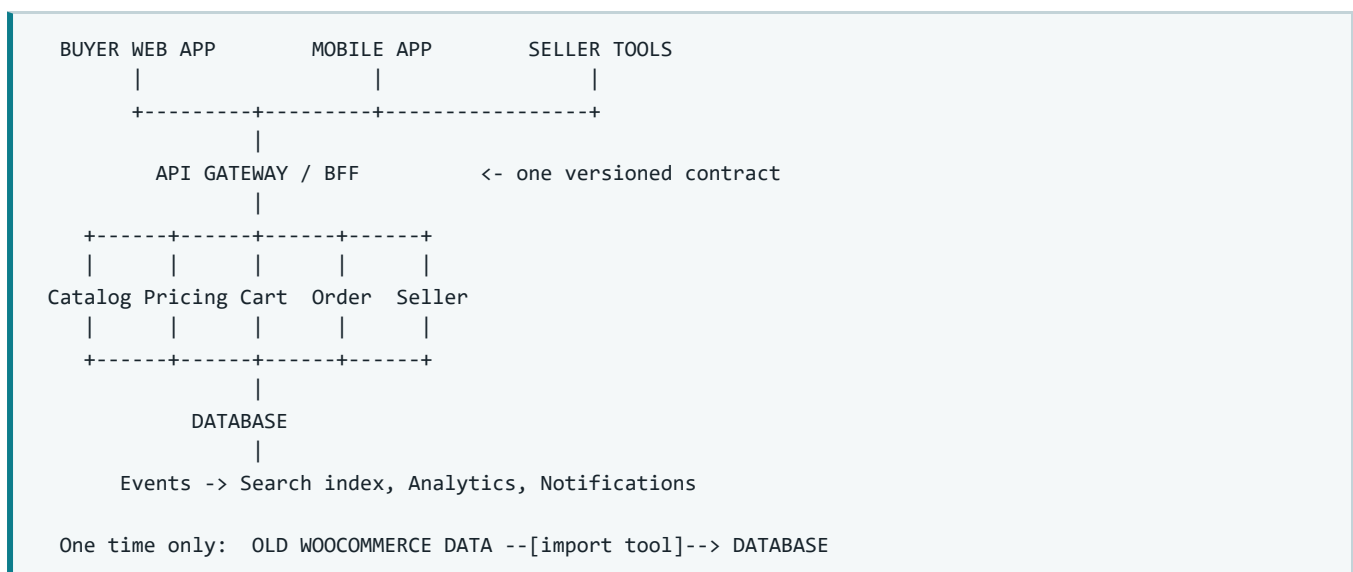
See section 7.2 for the section order. The action bar stays at the bottom.

15.4 Cart and checkout

Element	Content	Behaviour
Shipment group	Items, shipping fee, arrival date	One card for each fulfillment leg.
Free-shipping progress	"Add P120 more for free shipping"	The message is true. Otherwise the system hides it.
Savings panel	The applied discounts and the total saved	The buyer taps to see each line. See section 17.3 for the wording rule.
Address row	A closed summary	The buyer taps to open it. Only one row is open.
Payment row	Cash on Delivery, GCash, card	Cash on Delivery is a main option.
Order control	Place Order • P1,319	A tap control. It shows the exact amount. It is not a drag control.

16. System architecture

16.1 The shape of the system



Service	It owns	It does not own
BFF	The response shape for each client, the version, the authentication context	The price arithmetic and the order record
Catalog	Products, variants, attributes, categories, media, seller link	Campaign discounts
Pricing	Price lines, promotion rules, shipping inputs	The interface state
Cart	Selected items, quantities, shipment groups	The final financial record
Order	Approved totals, payment intent, status history	The search index
Seller	Verification, product approval, commission, payout, dispute	Buyer personalisation

FINANCIAL RULE

The client never calculates a total that the system trusts. The server sends a quote with a version number. The checkout accepts that quote, or the checkout returns a price-change answer.

16.2 Repository structure

```
apps/
  storefront/      # the web interface
  bff/             # the versioned API for the web app and the mobile app
  seller/          # the seller tools
  worker/          # background jobs, events, search index, notifications
packages/
  contracts/       # schemas and generated clients
  domain/          # money, price, cart, order, fulfillment rules
  ui/              # components and design tokens
  observability/   # logs, metrics, traces
tools/
  import-legacy/   # the one-time WooCommerce import tool
infra/
  migrations/      # restartable data jobs
docs/
  adr/             # architecture decision records
  runbooks/        # launch and rollback procedures
```

16.3 Money

```
export type Money = { amountMinor: number; currency: "PHP" };

export type PriceLine = {
  kind: "ITEM" | "DISCOUNT" | "SHIPPING" | "TAX" | "FEE";
  label: string;           // the text that the buyer reads
  amount: Money;
  fundingSource?: "PLATFORM" | "SELLER" | "PARTNER";
};
```

- Store money as an integer in minor units. P12.34 is 1234 .
- Do **not** use a decimal number for money. Decimal arithmetic loses value.
- Do **not** use a product name or a slug as a financial identifier.

16.4 API rules

```
GET /api/v1/products?category=home-lighting&sort=price-asc&cursor=...
200 {
  "items": [{
    "id": "prd_123",
    "slug": "solar-wall-lamp",
    "title": "Solar Wall Lamp",
    "seller": { "id": "sel_7", "verified": true },
    "price": { "amountMinor": 34900, "currency": "PHP" },
    "media": [{ "kind": "IMAGE", "url": "...", "alt": "..."} ]
  }],
  "nextCursor": "opaque-token",
  "queryState": { "sort": "PRICE_ASC", "filters": [] }
}

POST /api/v1/checkout
Idempotency-Key: 01J...
{ "cartId": "cart_abc", "quoteVersion": "qv_17",
  "paymentMethod": "GCASH", "shippingAddressId": "addr_9" }
```

Rule	Reason
Use an opaque cursor. Put the query state in the web address.	The back control works. The buyer can share the link. Section 6.4 needs this.
Require an idempotency key on checkout and on refund.	A retry after a network failure does not create a second order.
Give every quote a version.	The system finds a price change before it charges the buyer.
Use typed error codes.	The app can show the correct message for each cause.
Keep the mobile contract backward compatible.	The MySALS3 application continues to work during the change.

Use these answers for a checkout conflict:

- 409 PRICE_CHANGED — send a new quote. Mark the changed lines.
- 409 STOCK_CHANGED — name the unavailable quantity. Give an alternative.
- 422 ADDRESS_UNSERVICEABLE — keep the cart. Give the couriers that serve the address.
- 202 PAYMENT_ACTION_REQUIRED — send the gateway action. Do not create a second order.

16.5 Frontend rules

Subject	Rule
Rendering	Render the catalogue pages and the product pages on the server. A search engine must read them.

State	Put the filters, the sort order, and the cursor in the web address. Keep short-life interface state in the component.
Data	Use a generated typed client. Set the cache time from the business need for each resource.
Errors	Show the cause and the next action. Do not show a raw error code to the buyer.
Analytics	Record: search, filter change, product view, add to cart, quote change, each checkout step, payment result, return request.

17. The price and promotion engine

Automatic voucher selection is not an interface feature. It is a pricing problem. Design it in the first weeks. It is very difficult to add later.

17.1 The shape of a promotion

Field	Example	Purpose
scope	seller, SKU, category, shipment, cart	It selects the lines that change.
benefit	10 percent, P100, free shipping	It defines the price change.
stackGroup	seller-discount	It stops an incompatible combination.
fundingSource	SELLER	It protects the seller budget and the platform budget.
budget	P50,000; one for each buyer	It limits the campaign cost.
priority	higher saving, then earlier expiry	It makes the result deterministic.
explanation	"Seller voucher saved P100"	The buyer reads this. The support team reads this.

17.2 The calculation

```
function quoteCart(cart, context, rules): Quote {
  const eligible = rules.filter(r => r.isEligible(cart, context));
  const candidates = buildAllowedCombinations(eligible); // obey stackGroup
  const evaluated = candidates.map(set => evaluate(cart, set, context));
  const winner = stableSort(evaluated, bySavingThenPolicy)[0];

  return {
    version: hash(cart.version, rules.version, winner.inputs),
    lines: winner.priceLines,
    total: sumMoney(winner.priceLines),
    applied: winner.explanations,
    expiresAt: winner.expiresAt
  };
}
```

17.3 The wording rule

LEGAL WARNING

Do not write "Maximum vouchers applied" unless the engine tests every allowed combination. A false savings claim is a misleading statement. Philippine Republic Act 11967 covers this subject.

Safe text: "These discounts are applied." Then show the list.

17.4 Required tests

- Golden carts. Each test cart has one exact expected total.
- Property tests. A total is never below zero. A budget is never above its limit.
- Concurrency tests. Two buyers use the last voucher at the same time.
- Expiry tests. A voucher expires during the checkout.
- Reconciliation. The quote total and the order record agree.

18. Data migration from the old system

WooCommerce is the source of the old data. It is not part of the new system. The import runs one time. Then the team stops the old system.

18.1 The import record

```
type MigrationRecord = {
  entity: "product" | "variant" | "customer" | "order" | "review" | "seller";
  sourceId: string;           // the identifier in the old system
  targetId?: string;         // the identifier in the new system
  sourceUpdatedAt: string;
  checksum: string;
  status: "PENDING" | "MIGRATED" | "FAILED" | "VERIFIED";
  errorCode?: string;
};
```

The import job follows these 5 steps:

1. Read a batch by a stable source identifier range.
2. Transform the data. Then validate it.
3. Write by source identifier. A second run does not create a duplicate.
4. Store the checksum and the audit record.
5. Compare the counts, the sums, and a sample of records.

RULE

The import must be restartable. A failure at record 40,000 must not need a restart at record 1.

18.2 Web addresses and search engines

Control	Requirement
---------	-------------

Address inventory	Export every indexed web address before the team changes a route. Record the traffic for each address.
Redirect map	Map one old address to one new address. Do not send many addresses to the home page. Do not build a chain of redirects.
Metadata	Keep the page titles, the descriptions, the canonical addresses, and the product structured data.
Sitemap	List only the new canonical addresses.
Validation	Crawl the test site. Compare the status codes. Then watch the errors and the traffic after the launch.

18.3 Category cleaning

The current site has duplicate and overlapping categories. This is catalogue debt.

- Build a source-to-target category map. A person reviews the map.
- Do **not** copy the duplicate categories into the new system.
- Redirect the web address of each removed category.

18.4 What the team must not lose

- Customer accounts and seller accounts
- Order history and payment references
- Reviews and their product link
- Seller ownership of each product
- Voucher balances and loyalty balances
- Web addresses with traffic
- Mobile application compatibility, deep links, and push notifications

19. Tests and launch gates

19.1 Test coverage

Test type	Minimum coverage
Unit	Money arithmetic, promotion rules, stacking, commission, status changes, address mapping
Contract	Payment gateway, courier, notification service, mobile API compatibility
Integration	Cart storage, checkout idempotency, payment callback, stock reservation, refund, payout
End to end	Buyer, seller, support, and finance scenarios on realistic data
Accessibility	Automatic checks, and a manual keyboard, screen reader, zoom, and touch review
Performance	Full catalogue size, cold cache, low-cost device, slow network
Migration	Counts, sums, references, checksums, samples, restart, rollback

19.2 Security

- Keep administrative credentials on the server. The browser uses buyer APIs only.
- Check the signature and the timestamp of every incoming webhook.
- Store secrets in a secret manager. Rotate them.
- Send payment data to the gateway. Do **not** store card numbers.
- Record access to personal data. Remove secrets from the logs.

19.3 Launch gates

The team launches only when all 6 statements are true.

1. No financial reconciliation defect is open.
2. No order, review, seller link, or web address is lost without an explanation.
3. The payment success rate is equal to the old rate, or better.
4. The buyer, seller, support, refund, and payout procedures pass.
5. The team completed a rollback rehearsal.
6. No severity-1 defect and no severity-2 defect is open.

19.4 Dashboards for the launch week

- Purchase funnel by device, by traffic source, and by application version
- Quote-to-order differences and voucher rejection reasons
- Payment success, duplicate prevention events, refund time
- API errors, API latency, queue depth
- LCP, INP, page-not-found count, and search zero-result rate

20. How to build the system from the start

20.1 The build principle

THE PRINCIPLE

Build one complete path through all layers. Then make it wider. Do not build all of one layer first.

A team that builds all the screens first has no working system for many months. A team that builds one path can test a real purchase in week 6. The second team finds the difficult problems early.

20.2 The dependency rules

These rules stop the most common build mistakes. Follow the order.

Do not build this	Before this
Any screen	The design tokens and the base components
The cart	The pricing service and the Money type

The checkout	The quote version and the idempotency key
The promotion rules	The price line model and the funding source field
The search index	The category map and the attribute names
The data import	The final data model of the new system
The seller tools	The commission and payout rules
The launch	The rollback rehearsal

20.3 The 8 stages

Stage 1 — Foundation

- Create the repository. Add the lint, type, and test pipeline.
- Add the deployment pipeline and a health endpoint.
- Build the design tokens: colour, text, space, radius, and state.
- Build 10 base components: button, input, chip, card, sheet, dialog, tabs, badge, skeleton, toast.
- Add the logging, the metrics, and the error reports.

Exit test: a page with the base components deploys, and the pipeline blocks a bad change.

Stage 2 — Data model and contracts

- Define the entities: product, variant, seller, category, attribute, buyer, address.
- Define the Money type and the PriceLine type. See section 16.3.
- Write the API schemas. Generate the typed clients.
- Write the golden scenarios. See section 20.5.

Exit test: the team agrees on the entity names, and the schemas generate a client.

WARNING

Do not go to stage 3 with an unclear data model. A change to the data model at stage 6 costs 10 times more than a change at stage 2.

Stage 3 — Catalogue read path

- Build the catalogue service: product, variant, category, media.
- Build the list route and the product route. Render them on the server.
- Build the filters with counts. Build the sort control.
- Build the state preservation. Test the 6 conditions in section 6.4.
- Load a realistic quantity of test products. Do not test with 20 products.

Exit test: a buyer finds a product with a filter, opens it, and goes back to the same position.

Stage 4 — Price and promotions

- Build the pricing service. It returns price lines and a total.
- Build the promotion model. See section 17.1.
- Build the rules engine and the quote version.

- Build the shipping quote input for the delivery region.
- Run the golden carts. Every total must be exact.

Exit test: every golden cart returns the exact expected total, and the same input gives the same output every time.

Stage 5 — Cart and checkout

- Build the cart service and the fulfillment groups.
- Build the checkout page with progressive disclosure.
- Build the guest path.
- Add the idempotency key and the quote version check.
- Connect one payment method. Add Cash on Delivery.
- Test a retry, a timeout, and a duplicate submission.

Exit test: the team places a real paid test order, and a network retry does not create a second order.

MILESTONE

Stage 5 is the first vertical slice. The system can now take money. Every stage after this makes the slice wider.

Stage 6 — Orders and post-purchase

- Build the order record and the status history.
- Connect the courier for the tracking data.
- Build the order list and the order detail screens.
- Build the self-service return request.
- Build the refund path and the notification messages.

Exit test: a test order moves from payment to delivery to a refund, and the buyer sees each step.

Stage 7 — Seller and administration tools

- Build the seller sign-up and the verification.
- Build the product upload and the approval queue.
- Build the seller order view and the stock control.
- Build the commission calculation and the payout report.
- Build the dispute and return handling for the seller.
- Build the support tools: order search, refund, and note.

Exit test: a real seller completes sign-up, upload, sale, and payout without help from a developer.

WARNING

Teams frequently forget this stage in the plan. A marketplace without seller tools cannot operate. This stage is large. Give it a real estimate.

Stage 8 — Migration and launch

- Build the import tool. See section 18.
- Run the import on a test system. Compare the counts and the sums.
- Build the redirect map. Crawl the test site.

- Rehearse the launch. Rehearse the rollback.
- Launch to a small percentage of traffic. Then increase it.

Exit test: the 6 launch gates in section 19.3 all pass.

20.4 The design track

The design work runs at the same time as the system work. It does not wait.

Step	Work	It supports
D1	Tokens and base components	Stage 1
D2	Product card and list screen	Stage 3
D3	Product page	Stage 3
D4	Cart and checkout	Stage 5
D5	Order and return screens	Stage 6
D6	Seller screens	Stage 7
D7	Test with 5 to 8 Filipino buyers on a low-cost telephone	After stage 5

20.5 The golden scenarios

Write these 5 scenarios at stage 2. Each scenario has one exact expected result. The team runs them at every stage after stage 4.

1. One seller. Payment by GCash. One product discount and one shipping fee.
2. Two sellers. Two shipments. Two arrival dates. One partial refund.
3. A guest buys. The buyer creates an account later. The order joins the new account.
4. Four voucher types: expired, seller-funded, non-stackable, and limited by usage.
5. A product with variants, one unavailable variant, existing reviews, and an old web address.

20.6 The first 10 working days

Day	Action	Output
1	Get access to the old system, the hosting, the analytics, and the payment accounts. Name an owner for each.	<code>access-matrix.md</code>
2	Export the plugin list and the custom code from the old system. This shows the true feature list.	<code>current-features.md</code>
3	Map the integrations: payment, courier, notification, analytics, accounting.	<code>integration-map.md</code>
4	Measure the data volumes. Take a sample of each entity.	<code>data-catalog.md</code>

5	Export the top web addresses, the traffic, the conversion rate, and the device mix.	seo-baseline.csv
6	Record the seller, support, and finance procedures with those teams.	feature-list.csv
7	Write the golden scenarios with the exact expected totals.	golden-scenarios.md
8	Build the repository, the pipeline, and the health endpoint. This is stage 1.	A deployed skeleton
9	Build the token layer and 3 base components.	A component page
10	Present the data model, the risks, and the estimate.	ADR-001 and a backlog

FIRST CODE CHANGE

Make the first change small. Build the repository rules, the test pipeline, the contracts package, and a health endpoint. Do not start with the home page.

21. What to expect

This section gives realistic expectations. It protects the team from a wrong plan.

21.1 Where the effort goes

Many people think that a marketplace is mostly buyer screens. This is not correct. The table below shows the approximate effort for a marketplace build.

Part of the system	Approximate effort	Note
Buyer screens (web and mobile)	25 to 30 percent	The visible part. It is not the largest part.
Price, promotion, and order logic	20 to 25 percent	The most difficult part. It is invisible.
Seller and administration tools	25 to 30 percent	Teams forget this part most often.
Migration, integrations, and launch	15 to 20 percent	The work grows near the launch.
Support and finance tools	5 to 10 percent	Small, but the launch fails without it.

EXPECT THIS

The team will spend more time on the parts that the buyer never sees. A plan that gives 70 percent of the time to screens is wrong.

21.2 Time

The table gives a range for a first launch with the 8 stages. The range depends on the team size. These are estimates, not promises. The team gives a real estimate after day 10.

Team size	Time to first launch	Condition
1 to 2 developers	9 to 14 months	Only with a reduced first release. See section 21.3.
3 to 5 developers	6 to 9 months	With one designer and one test engineer.
6 to 10 developers	4 to 6 months	With clear service boundaries and a product owner.

These conditions change the time:

- A payment integration takes 2 to 6 weeks. The certification of the gateway controls the date.
- A courier integration takes 2 to 4 weeks for each courier.
- The mobile application needs its own release cycle. The application store review takes days.
- The seller tools take longer than the estimate in almost every project.

21.3 The smallest useful first release

A small team must reduce the first release. Cut the scope, not the quality. Keep these items. Remove the rest.

Keep in the first release	Remove from the first release
Browse, search, filters, product page	Vector search and recommendations
Cart, guest checkout, one online payment, Cash on Delivery	Many payment methods
Order tracking and return request	Loyalty points and referral programs
Seller sign-up, product upload, order view, payout report	Advanced seller analytics
One correct price with a discount breakdown	Complex campaign types
Support order search and refund	A full support ticket system

21.4 The difficult parts

These 6 parts cause the most delay. Give them the most experienced people.

Rank	Part	Why it is difficult
------	------	---------------------

1	Price and voucher combinations	The quantity of combinations grows quickly. The result must be the same every time. Money errors are not acceptable.
2	Multi-seller orders	One order becomes many shipments, many commissions, and many partial refunds.
3	Payment states	A payment can succeed at the gateway and fail at the site. The system must not lose the money or duplicate the order.
4	Data migration	The old data has errors. The team finds them only during the import.
5	Search relevance	Correct results need clean categories, attributes, and synonyms. This is manual work.
6	Seller payouts	The arithmetic must agree with the finance records for every period.

21.5 What will go wrong

These events happen in most marketplace projects. Plan for them. They are not a surprise.

- The old data has products without a category, orders without a seller, and duplicate customers.
- The plugin list shows features that nobody documented. The team finds them at stage 7.
- The first payment integration takes longer than the estimate.
- A seller reports that a commission is wrong. The cause is a rounding rule.
- The mobile application breaks because an API field changed.
- The search gives bad results because the categories are not clean.
- A stakeholder asks for a new feature at stage 6. This moves the launch date.

21.6 After the launch

A new system does not give better numbers on day one. Expect this pattern.

Period	What to expect
Week 1	Small defects appear every day. The team fixes them each day. Search engine traffic falls. This is normal after a route change.
Weeks 2 to 4	The defect rate falls. The search engine re-reads the site. The traffic starts to return.
Months 2 to 3	Search engine traffic returns to the old level, if the redirect map is correct. The conversion rate starts to improve.
Months 4 to 6	The team sees the true effect of the design rules. Now the team can measure an improvement.

EXPECT THIS

Search engine traffic falls after a launch, even with a correct redirect map. A fall of 10 to 30 percent for 4 to 8 weeks is normal. A fall that continues after 3 months shows a redirect problem.

21.7 What success looks like

Measure the old system before the launch. Then compare. Do not use an industry average.

Measurement	Target after 6 months
-------------	-----------------------

Cart abandonment	Lower than the old rate. The price rules in section 5 cause this.
Search zero-result rate	Below 5 percent of searches.
Checkout completion	Equal to the old rate at launch. Better after 3 months.
Page speed (LCP, 75th percentile)	Below 2.5 seconds on a low-cost Android telephone.
Return to Sender rate	Lower than the old rate. The delivery date and the address rules cause this.
Repeat purchase rate	Higher than the old rate. The return process in section 10 causes this.
Support contacts about price	Near zero. The price breakdown in section 5.2 causes this.

21.8 What the team needs from the owner

The build stops without these answers. Get them in the first 10 days.

- Which business result defines success? Which number does the team measure?
- Does the first release include the mobile application, the seller tools, and the finance tools?
- Which system holds the true price and the true stock during the change?
- Which web addresses and which application versions must continue to work? For how long?
- What is the approved launch window and the approved rollback window?
- Who approves a new feature request during the build?

THE LARGEST RISK

The largest risk is not the technology. The largest risk is a change of scope during the build. Name one person who approves a scope change. Record every change and its cost in time.

22. References

Each source has a reliability level. Use level A for a decision. Do not use level C for a decision.

Level	Meaning
A	A standard, a law, or official vendor documentation. Use it for a decision.
B	A research organisation with a published method. Read the method first.
C	A blog or a marketing page. Use it for an idea only. Never for a decision.

Level	Source	Use in this document
A	Republic Act 11967, Internet Transactions Act lawphil.net/statutes/repacts/ra2023/ra_11967_2023.html	Merchant identity, price transparency, consumer redress. Sections 9, 14, 17.3.
A	Philippine Trustmark, Department of Trade and Industry trustmark.dti.gov.ph	The verification mark on the seller card.

A	W3C Web Content Accessibility Guidelines 2.2	Contrast ratio, focus mark, drag alternative. Section 13.
A	W3C Design Tokens Format Module designtokens.org	The 3-level token structure. Section 11.1.
A	EU Digital Services Act	Dark pattern prohibition. It applies to EU users. A lawyer must confirm the effect on Sals3.
B	Nielsen Norman Group, e-commerce guidelines	Filter design and product page structure. Sections 6 and 7.
B	Baymard Institute, checkout research	The causes of an abandoned cart. Use the causes. Do not use the percentages as a promise.

LEGAL NOTE

This document is not legal advice. A Philippine lawyer must review the compliance sections before the launch.

HOW TO USE THIS DOCUMENT

Use section 3 as the decision record. Use sections 4 to 15 to design the screens. Use sections 16 to 19 to write the code. Use section 20 for the build order. Use section 21 to set the expectations with the owner.

Replace an estimate with a measurement when the team gets the data.